Uppgift 5 – Robot Wars

Mål:

Målet med denna uppgiften är att skapa AIs som tar olika beslut beroende på omständigheter runt omkring dem, istället för att exekvera samma sak varje frame som vi tidigare gjorde med steering behaviours. För att uppnå detta ska vi lära oss att implementera ett Decision Tree och en State Machine.

Generella krav:

- Det går bra att använda TGE, er egen motor eller ett tredje parts bibliotek (t.ex. raylib eller SDL).
- Programmet ska kompilera utan kompileringsvarningar på warning level 4 och med ”treat warnings as errors” satt till ”yes”.
- Trådar som används ska joinas korrekt när programmet stängs av.
- Actors får inte flimmra/hacka, deras movement ska vara smooth. (Inom rimliga gränser. Om en actor gör ett pyttelitet ”hack” när den når sitt mål så är det ok, men den ska inte kontinuerligt bete sig glitchigt. Är du osäker om din movement är tillräckligt smooth så poke:a mig innan inlämning så gör jag en bedömning.)

Uppgift-specifika krav:

- Ert Decision Tree ska ha minst två Decision Nodes.
- Er State Machine ska ha minst tre states. Ni får inte använda ett Decision Tree för att avgöra vilket state State Machine:en ska ha.
- Death är inte en Decision Node eller ett State, utan är något som bör hanteras i Actor-klassen. Death räknas alltså inte in i era två Decision Nodes eller i era tre State Machine states.

Uppgiften:

- När spelet startar ska man se två actors. En ska styras av ett Decision Tree och den andra av en State Machine. De ska ha samma beteende som är följande:
- De ska undvika att köra in i och kollidera med väggar. Världen behöver inte vara komplicerad, ok om den bara är en tom rektangel.
- Era actors ska ha HP. Om deras HP faller under en viss gräns så anses de vara ”wounded” och ska då söka sig till en health pool som finns i leveln. När de står nära health pool:en så ska de gradvis få tillbaka HP. Det ska gå att se hur mycket HP en actor har, dvs HP ska renderas på något sätt.
- Om de inte är wounded så ska de kolla om de har vision på den andra actorn. Om de inte har det så ska de bara vandra/idle:a omkring i leveln. Om de har vision så ska de jaga och försöka skjuta den andra actorn. Ni kan implementera vision väldigt enkelt, kan bara vara en distance

check t.ex. Metrics som bullet speed, fire rate, bullet spread etc är upp till er. Tänk på hur dessa kan påverka inte bara balansen i spelet utan även vilken känsla AIn ger spelaren!

- Actors måste göra mer skada än vad health poolen kan heala. Annars kan ni lätt hamna i ett dödläge där båda actors står i health poolen och skjuter på varandra men ingen tar skada.
- När en actor dör så ska den respawna efter ett par sekunder.
- Om ni vill så får ni göra en mer komplicerad värld som har mer geometri och ha mer komplicerad vision check, typ ray casts. Ni får också ha mer komplicerat beteende på era AIs om ni vill. Så som uppgiften är beskriven är bara minimum. Men det är ändå ganska mycket att implementera.
- Implementerings-tips:
    - Börja med actorn som har ett Decision Tree. Implementera en feature i taget, t.ex implementera först dess normal-läge, dvs wandering + wall avoidance.
    - Implementera sedan HP och health poolen. Gör så att actorn kan ta skada via någon hårdkodad hotkey, t.ex spacebar.
    - Implementera shooting. Fixa så att State Machine actorn finns i leveln men inte gör någonting, bara så att Decision Tree actorn har ett target.
    - När Decision Tree actorn är klar kan ni ”disable:a” den så att den finns i leveln men inte gör någonting och börja implementera State Machine actorn.
    - När båda era actors är klara, prova att enable:a båda och se dem battle it out. >:)

Inlämning:

- På Omniway. Parprogrammering går fint, skriv i så fall i er inlämning vem ni har parprogrammerat med.
- En mapp för Exe och en för Source.
- Ni ska lämna in antingen en Release-exe, eller en Debug-exe och en Retail-exe. Detta för att debug drawing är disabled i Retail.
- Inga skräpfiler som .vs-mappar eller liknande får förekomma. Var noga med att köra Clean på din solution innan du lämnar in.